7
Amazon Cloud Security Services

Information security is protecting data and resources including sensitive information, computer systems, networks, and so on from unauthorized access, disruption, theft, or damage. Security also prevents attacks with a quick and effective response if an attack does occur. Effective security requires a combination of people, technologies, and processes.

Amazon Cloud Security protects data and resources on the AWS cloud platform. AWS provides a comprehensive cloud security model and a set of services to help customers secure their cloud-based applications and infrastructure. In this chapter, we will discuss the following topics:

- **AWS cloud security model**: The customer is responsible for the security of the cloud, and the provider is responsibility for security in the cloud.
- **AWS Identity and Access Management**: A cloud security service that enables customers to centrally manage access to AWS resources and services. Access is the first line of security defense, and this is where we practice the *least privilege* and *minimum attack surface* security principles.
- **AWS infrastructure security**: The security of Amazon cloud infrastructure, including cloud *endpoint* resources such as EC2, RDS, S3, and cloud *network* resources such as VPCs and subnets. This is where we practice the *zero-trust* security principle and the *multi-layer protection* security principle.
- **AWS data security**: Cloud data protection, including data encryption at rest and in transit, and encryption key management. This is where we enforce the *most encryption* security principle.
- **AWS cloud logging and monitoring**: Tracing all cloud assets and activities, all the time. This is where we enforce the *traceability* security principle.
- **AWS cloud threat detection and remediation**: Threat detection and remediation in the cloud. We will conduct a case study about *a threat auto-detection and auto-remediation ecosystem*.

We will start our journey with the AWS cloud security model.

Amazon cloud security model

The **AWS shared responsibility model** is about the responsibilities of AWS and its customers. AWS is responsible for the *security of the cloud*, including the physical infrastructure, network, hypervisor, and so on that supports the customer's applications and data. The customer is responsible for *security in the cloud*, including customers' data, applications, and other configurations that are hosted on the AWS infrastructure, such as access management, firewall configurations, data encryptions, and so on.

Based on the shared responsibilities, AWS provides a multi-layered security model to protect customer data and resources in the cloud, including the following layers:

- **Physical/hardware**: AWS data centers are designed and managed to comply with security standards and regulations by employing physical security equipment such as access control systems, surveillance cameras, and perimeter fencing to prevent unauthorized access. These are Amazon's responsibility and AWS provides an on-demand service called **AWS Artifact** that offers access to a central repository of physical security and compliance reports audited by third parties.
- **IAM**: AWS gives customers the ability to manage user access and permissions to AWS services and resources. With IAM, customers can create and manage IAM users, groups, and roles to control access to AWS services.
- **Network security**: AWS provides network security measures such as VPCs, SGs, and NACLs to ensure that data traffic between resources is protected and controlled. Customers are responsible for configuring the cloud VPCs, SGs, and NACLs.
- **Data encryption**: AWS offers multiple encryption options, including server-side encryption, client-side encryption, and **Key Management Service** (**KMS**), to protect data at rest and in transit.
- **Compliance and auditing**: AWS provides compliance and auditing capabilities to meet various industrial regulatory requirements and standards, such as HIPAA, SOC 2, PCI DSS, and ISO 27001.
- **Application security**: AWS offers security tools and services, such as **AWS Web Application Firewall** (**WAF**) to protect against web application attacks, and **AWS Shield** to protect against **Distributed Denial of Service** (**DDoS**) attacks.

AWS's multi-layered cloud security model provides a comprehensive approach to securing cloud resources and allows customers to build and deploy applications securely in the cloud. Let's explore Amazon IAM first.

Amazon IAM

IAM manages resource identity and accessibility, including authentication, authorization, and accounting.

Authentication is authenticating an identity to access an information system. One of the important security features of IAM is that it supports **Multi-Factor Authentication** (**MFA**), which requires users to provide a second form of authentication, such as a one-time token, or biometric identity, in addition to their username and password. IAM also supports *identity federation*, which allows customers to integrate their existing identity management systems with AWS, enabling users to sign into AWS using their existing credentials, such as Google or Meta logins, and so on.

Authorization is the user's permission once they are authenticated in the system. With IAM, customers can create and manage AWS users and groups, and define permissions that grant or restrict access to specific AWS resources. Authentication defines who can perform which actions on what resources. The *Who* can be a user, a group, or a role. IAM supports **Role-Based Access Control** (**RBAC**), which allows customers to assign permissions to roles rather than individual users. The *What* is cloud resources including EC2, EBS, EFS, S3, VPC, and so on, and the *Which* is the actions that the *Who* can take on *What*.

Auditing is logging and tracking user activities in the AWS cloud. We will explore more about AWS auditing in the *Monitoring and logging* portion of this chapter.

IAM policies

In AWS IAM, *every permission is a policy*. IAM permissions are defined with an AWS policy – a JSON document. The following is a sample policy:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "AllowReadAccess",
            "Effect": "Allow",
            "Action": [
                "s3:*"
            ],
            "Resource": [
                "arn:aws:s3:::bucket1",
                "arn:aws:s3:::bucket1/*"
            ]
        }
    ]
}
```

The preceding policy defines the permissions as "*any S3 actions against an S3 bucket named bucket1*." If we attach this policy to a user, then the user will have all the permissions to act against *bucket1*. Since this policy defines permissions on specific resources, it is a *resource-based IAM policy*. There is another type of policy that defines permissions on the principal (user, group, role, identities), and we call it an *identity-based IAM policy*. The following example shows an identity-based IAM policy:

```
{
    "Version": "2012-10-17",
    "Statement": {
        "Effect": "Deny",
        "Action": "*",
        "Resource": "*",
        "Condition": {
            "NotIpAddress": {
                "aws:SourceIp": [
                    "10.10.0.0/16",
                    "8.8.0.0/16"
                ]
            }
        }
    }
}
```

We will create several IAM policies and see how they are used in managing S3 objects

1. Log in to the AWS console as an admin, and create the following four policies:

- **Policy A "userjohn"**: This policy defines read-only resource permissions and assume-role permissions:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": [
                "iam:Describe*",
                "iam:GetAccountAuthorizationDetails",
                "iam:GetPolicy",
                "iam:GetRole",
                "iam:GetRolePolicy",
                "iam:GetUser",
                "iam:GetUserPolicy",
                "iam:List*",
                "logs:Desc*",
                "logs:Get*",
                "logs:List*",
```

```
                "s3:ListAllMyBuckets",
                "s3:ListBucket",
                "s3:PutAccountPublicAccessBlock",
                "s3:PutBucketOwnershipControls",
                "s3:PutBucketPublicAccessBlock",
                "sts:AssumeRole"
            ],
            "Resource": "*",
            "Effect": "Allow"
        }
    ]
}
```

- **Policy B "switchrole"**: This policy defines `list bucket` and `get bucket` permissions for all S3 buckets, and `list, get (download) objects` permissions for an S3 bucket named `03252023-bucket1`:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": [
                "s3:GetBucketPolicy",
                "s3:ListBucket"
            ],
            "Resource": "*",
            "Effect": "Allow"
        },
        {
            "Action": [
                "s3:GetObject",
                "s3:ListObjects",
                "s3:ListBucket"
            ],
            "Resource": [
                "arn:aws:s3:::03252023-bucket1",
                "arn:aws:s3:::03252023-bucket1/*"
            ],
            "Effect": "Allow"
        }
    ]
}
```

- **Policy C "trustpolicy"**: This allows the user `userjohn` to assume the role assigned to it:

```
{
    "Version": "2012-10-17",
```

```
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::317332158300:root"
            },
            "Action": "sts:AssumeRole",
            "Condition": {}
        },
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::317332158300:user/userjohn"
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
```

- **Policy D "bucket2-permission"**: This gives the `switchrole` role `list bucket` and `get/put objects` permissions on `03252023-bucket2`:

```
{
    "Version": "2008-10-17",
    "Statement": [
        {
            "Sid": "S3Write",
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::317332158300:role/
switchrole"
            },
            "Action": [
                "s3:GetObject",
                "s3:PutObject"
            ],
            "Resource": "arn:aws:s3:::03252023-bucket2/*"
        },
        {
            "Sid": "ListBucket",
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::317332158300:role/
switchrole"
            },
            "Action": "s3:ListBucket",
```

```
                "Resource": "arn:aws:s3:::03252023-bucket2"
            }
        ]}
```

2. As the AWS account admin, create a user, a role, and some S3 buckets:

   I. Under IAM, create a user named `userjohn` with AWS console access, and assign permission policy A, `userjohn`, to the `userjohn` user.

   II. Create a role named `switchrole` with permission policy B, `switchrole`, and trust policy C, `trustpolicy`, as shown in *Figure 7.1*.

   III. Under the S3 console, create two buckets named `03252023-bucket1` and `03252023-bucket2`:

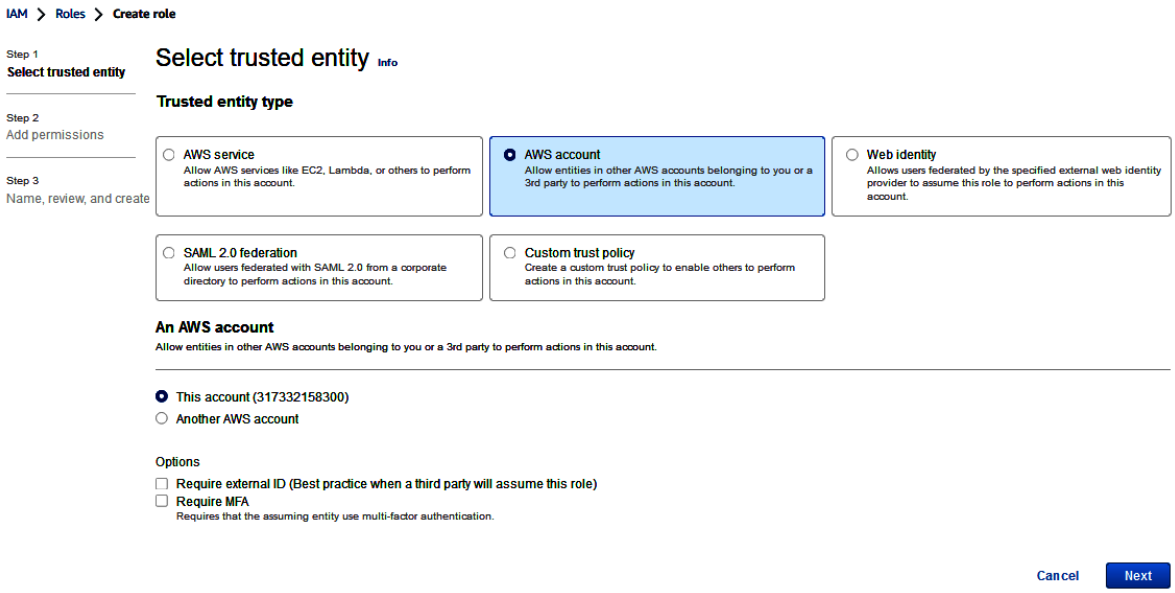


Figure 7.1 – Create an IAM role called "switchrole"

3. **Log in to the AWS console as the userjohn user**:

Based on the authorization defined in the policy, `userjohn` will have read-only permission in the EC2 dashboard or S3 console, as shown in *Figure 7.2*:

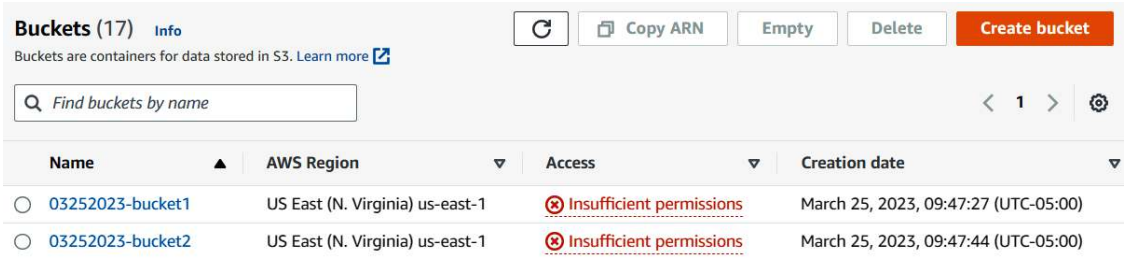


Figure 7.2 – userjohn has read-only permissions

Go to the S3 console and try to upload an object to the two S3 buckets. It will fail since the user has read-only permissions to the S3 bucket. This is shown in *Figure 7.3*:

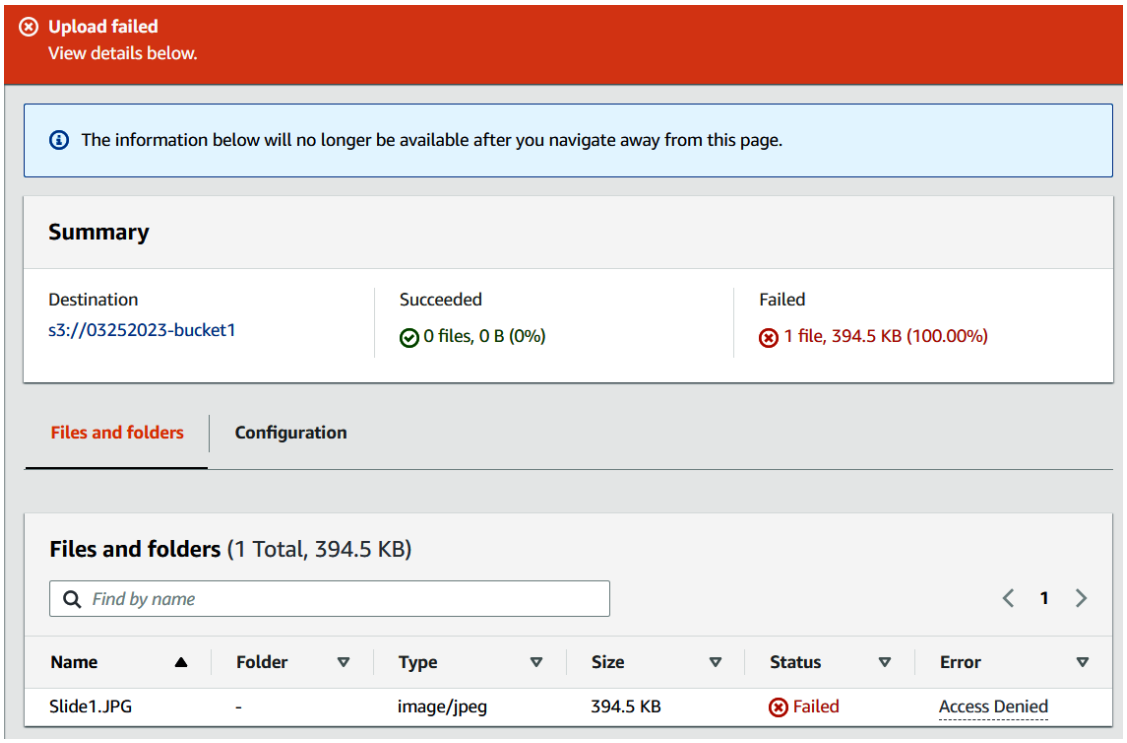


Figure 7.3 – userjohn uploading an image to bucket1 failed

4. Switch role and upload the object to **bucket1**:

   Go to the AWS console and click **userid** in the top-right corner; select **Switch Role**. Fill in the AWS account number and enter `switchrole` as the role name, as shown in *Figure 7.4*:

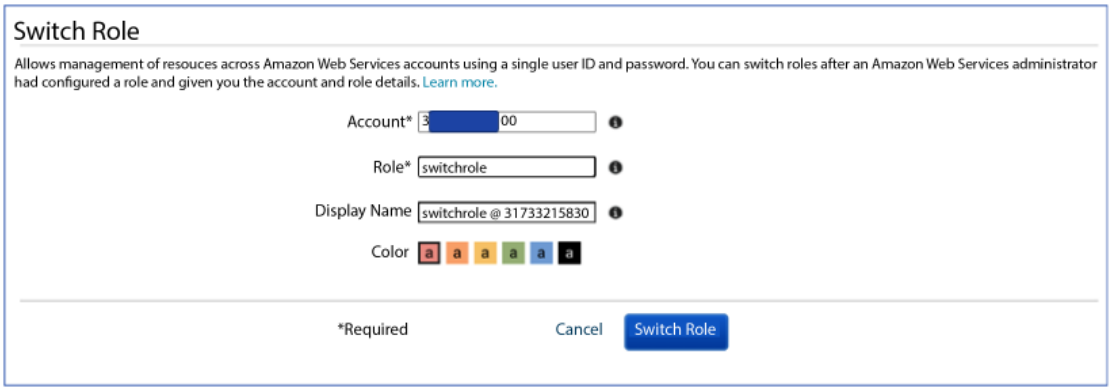


Figure 7.4 – Switch role from userjohn to switchrole

   After switching to the role, you will be able to upload objects to the `03252023-bucket1` S3 bucket since `switchrole` has the right permissions assigned to it. However, you still cannot upload objects to the `03252023-bucket2` S3 bucket since no permissions are granted to it. We will fix that with the admin account.

   Open another window, log in to the AWS console as `admin`. Go to bucket `03252023-bucket2`, and add policy D, `bucket2-permission`, to it

- Go back to the AWS console window where you're logged in as `userjohn`.

Upload an object to bucket `03252023-bucket2` again, and now it succeeds since `03252023-bucket2` is assigned the `bucket2-permission` policy, which permits `switchrole` to put objects. This is shown in *Figure 7.5*:

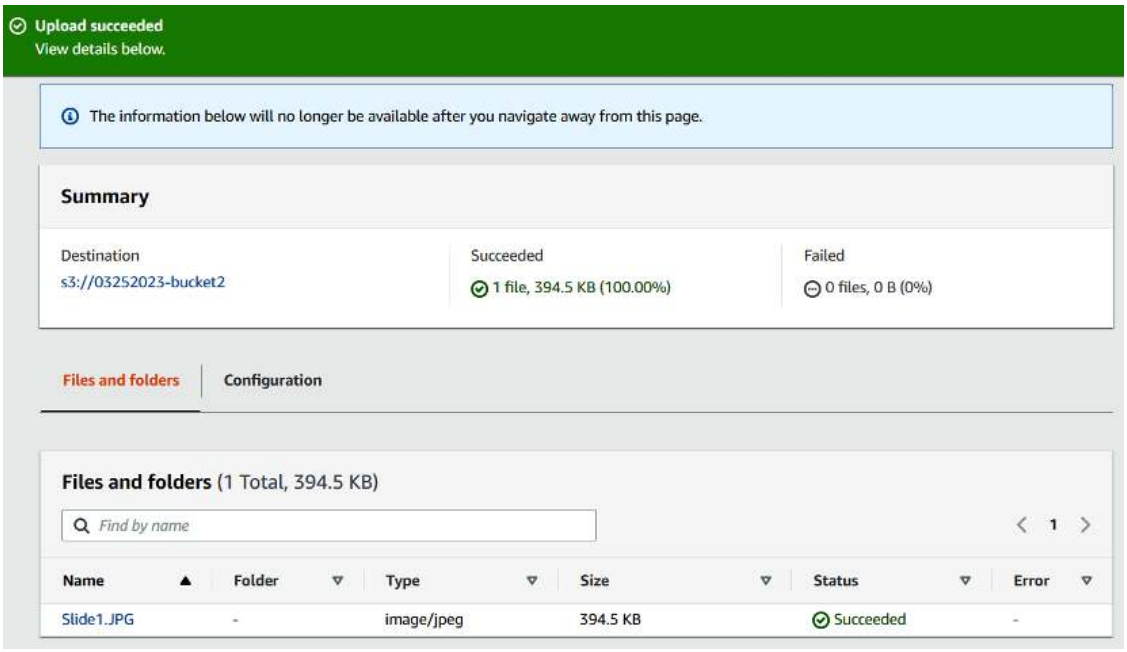


Figure 7.5 – userjohn uploading to bucket2 succeeds

From the preceding process, we can see that all the IAM permissions are defined as policies, so we can assign permissions to users, groups, roles, and S3 buckets, and we can switch roles to get different permissions for AWS resources.

In summary, AWS IAM provides customers with a powerful set of tools for managing AWS cloud resources. In the next section, we will further explore AWS infrastructure security.

AWS infrastructure security

The basic unit of AWS infrastructure is the *12-digit AWS account* that we have been logging in to and configuring our cloud services with. When you have many AWS accounts, it is necessary to have an infrastructure hierarchy – called **AWS Organizations** – to manage these accounts. In this section, we will first discuss the AWS resource organization hierarchy and security policies, and then inspect security for the AWS basic infrastructure components: EC2, S3, VPC, databases, and many others.

AWS Organizations

AWS Organizations consolidates multiple accounts into a central management unit, to manage business budgets, security, and compliance. With AWS Organizations, you can do the following:

- Automate AWS account creation and management using AWS APIs
- Consolidate billing and perform cost management across many AWS accounts, at various levels of your organization

With IAM policies, you can manage permissions for AWS users, groups, or roles, but you cannot restrict the AWS account root user. However, with Organizations, you can use **service control policies** (**SCPs**) to manage access to AWS services for individual AWS accounts, or for groups of accounts in an **Organization Unit** (**OU**), as shown in *Figure 7.6*:

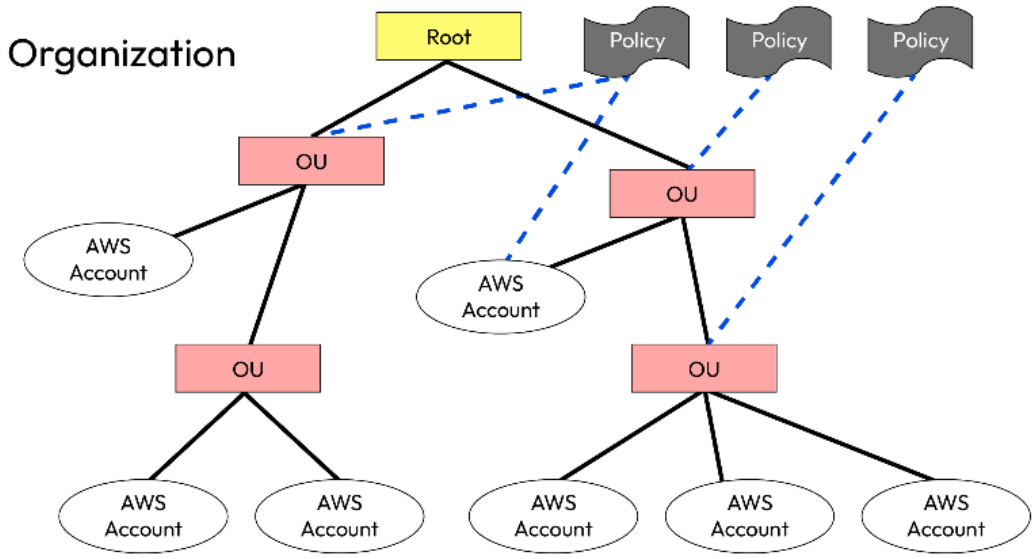


Figure 7.6 – AWS Organizations

Having reviewed security at the OU and account levels, now let's look at security management at the resource and service levels.

AWS resource security

We will discuss AWS resource security in the following basic domains:

- AWS compute security, mainly EC2
- AWS storage security, mainly S3

- AWS network security (VPC)
- AWS data security, including databases, big data, and machine learning

Amazon compute security

We discussed EC2 in *Chapter 1*. EC2 instances are virtual machines that customers can launch in the AWS cloud to run applications and services. The key security features of AWS EC2 include the following:

- **Instance isolation**: EC2 instances are isolated from each other using virtualization technology. This ensures that a security breach in one instance does not affect other instances running on the same host.
- **EBS encryption**: EBS is a block-level storage service used by EC2 instances. EBS volumes can be encrypted to ensure that data is protected at rest.
- **AMIs**: AMIs are pre-configured templates for EC2 instances that include operating systems, applications, and other software. Customers can create their own custom AMIs to ensure that their instances are pre-configured with the necessary security settings.
- **IAM**: IAM enables customers to create IAM policies that control who can launch and terminate instances, as well as access their instances using the AWS Management Console.
- **SGs**: EC2 instances are secured using security groups, which act as virtual firewalls. Customers can define inbound and outbound rules for each security group to control access to their instances.
- **Key pairs**: EC2 instances are accessed using key pairs, which consist of a public and private key. Customers can use key pairs to securely log in to their instances.

By following best practices and using these features, customers can secure and protect compute resources from unauthorized access.

Amazon storage security

We discussed S3 in *Chapter 2*. Amazon S3 is a cloud-based storage service that is designed to provide secure and scalable storage for businesses and developers. The key security features of AWS S3 are as follows:

- **Access controls**: S3 enables customers to set access controls on their buckets, including bucket policies and access control lists. We discussed bucket-level permission policies earlier in this chapter.
- **MFA delete**: This requires an authentication code from an MFA device before allowing the deletion of objects in an S3 bucket.
- **Versioning**: This supports multiple versions of objects to protect against the accidental deletion or modification of objects.

- **Encryption**: S3 supports server-side encryption for data at rest. Customers can choose to encrypt their data using different encryption methods.
- **Access logging**: S3 provides access logging, which enables customers to log all requests made to their buckets, to help with audit and compliance requirements.
- **Cross-region replication**: S3 provides cross-region replication, which enables customers to replicate their data across multiple regions. This can help with disaster recovery and data resilience.

Amazon storage provides a range of security features and controls to secure cloud storage from unauthorized access.

Amazon network security

We discussed VPC in *Chapter 3*. Amazon VPC is a cloud network service that allows customers to create an isolated network within the AWS cloud, to deploy cloud resources such as Amazon EC2 instances, databases, and so on. Here are some of the key security features of AWS VPC:

- **Network isolation**: VPC enables customers to create a private network within the AWS cloud. This network is isolated from the public internet and other VPCs, providing an additional layer of security.
- **Private subnets**: VPC allows customers to create private subnets within their VPC that have no direct access to the internet. Resources in private subnets can only communicate with other resources within the VPC.
- **Network ACLs**: VPC provides network **Access Control Lists** (**ACLs**), which are stateless and allow customers to control inbound and outbound traffic at the subnet level. Network ACLs can be used in conjunction with security groups to provide layered security.
- **VPN connections**: VPC provides the ability to establish VPN connections between the customer's on-premises network and their VPCs, to securely access resources within their VPC from their on-premises network.
- **VPC Flow Logs**: This enables you to record the traffic flowing to and from resources in VPCs, to analyze, troubleshoot, and audit network traffic flows.

AWS VPC and VPC peering secure cloud networks in multiple layers, including VPC routers and routing tables, firewalls (NACLs and SGs), and monitoring and logging tools (such as Flow Logs). Third-party tools such as Palo Alto virtual firewall appliances can also be deployed into VPCs for secure traffic management.

Amazon data security

We discussed AWS databases in *Chapter 4*, data analytics in *Chapter 5*, and ML in *Chapter 6*. These data services provide built-in security features to help customers protect their data stored in the cloud, including the following:

- **Encryption**: AWS provides various encryption options to help customers protect their data at rest and in transit. For example, RDS supports encryption at rest using **AWS Key Management Service** (**KMS**), DynamoDB supports server-side encryption, and Kinesis supports the encryption of data in transit using SSL/TLS, and also provides the option to encrypt data at rest using server-side encryption or customer-managed keys with AWS KMS.
- **Access controls**: AWS database services provide various access control mechanisms, such as IAM policies, database-specific access controls, and VPC security groups. IAM policies can be used to restrict access to Glue, EMR, Athena, and other resources. These access controls enable customers to restrict access to their databases and data.
- **Auditing and logging**: AWS database and data analytics services provide various auditing and logging features, such as AWS CloudWatch, CloudTrail, RDS audit logs, DynamoDB streams, and EMR logs. These features allow customers to track and monitor database activity and can be used for compliance and auditing purposes.
- **Backup and recovery**: AWS database and data analytics services provide backup and recovery features to help customers protect against data loss. For example, Amazon RDS provides automated backups and snapshots, and Amazon DynamoDB provides point-in-time recovery.
- **Data replication**: AWS database and data analytics services provide various data replication options, such as multi-AZ deployments, read replicas, and global tables. These replication features can help improve database performance and availability while also providing data redundancy and disaster recovery capabilities.
- **Compliance**: AWS database services comply with various industry standards and regulations, such as HIPAA, PCI DSS, and GDPR. Customers can use these services to help meet their compliance requirements.

Overall, AWS database services provide a range of security features and controls to help customers protect their data stored in the cloud. In the next section, we will discuss data encryption.

Amazon data encryption

Data encryption is a must for securing sensitive data in the cloud. As we have discussed, almost all the AWS cloud data services provide data encryption. In this section, we will spend time introducing AWS KMS and explain how it is leveraged in S3 object encryption using an example.

KMS is a fully managed service to manage encryption keys. It is designed to simplify the process of creating and managing encryption keys, whether they are stored in Amazon S3, EBS, RDS, or other services. AWS KMS provides the following:

- **Centralized key management**: With AWS KMS, you can centrally manage encryption keys used to protect your data across multiple AWS services and applications.
- **Customizable key policies**: You can set fine-grained access controls on your encryption keys to define who can use them and under what conditions.
- **Encryption key creation**: AWS KMS enables you to create new encryption keys, import your own keys, and manage the lifecycle of your keys.
- **Integration with other AWS services**: AWS KMS is tightly integrated with other AWS services, making it easy to use encryption to protect your data across your entire cloud environment.
- **Auditing and compliance**: The AWS KMS service provides detailed logging and auditing capabilities that help you meet regulatory and compliance requirements.

Now let us dive into the encryption process of an S3 object, to understand how KMS is leveraged in securing AWS data in the cloud:

1. **Create a KMS key**:

   The *AWS KMS master key* is used to generate, encrypt, and decrypt *data keys* that will be shared with other AWS services, such as S3 and EC2, to encrypt the actual data stored in an S3 bucket and on EBS volumes.

   Log in to the AWS Management Console, use the IAM service to add the user group `keyadmin` as key administrators who will *manage access* to the encryption key, and add another group, `keyusers`, for key users who will *use the key* to encrypt and decrypt data.

   Go to the AWS KMS console and then **Customer Managed Keys** | **Create Key**:

- For **Key type**, choose **Symmetric**.
- For **Alias**, enter the name of the master key, `mk`.
- For **Key administrators**, select **keyadmin**.
- For **Define key usage permissions**, select **keyusers**.
- At the bottom of the **Review** page, choose **Finish**.

2. **Create an S3 bucket and upload an encrypted object**:

   Go to the S3 console and create a bucket named `data`. Click the bucket and choose the **Properties** tab. In the **Default encryption** section, notice that the setting is currently disabled for the bucket. We will upload a file to the bucket and store it as an encrypted object:

   i. At the top of the page, go to the **Objects** tab. Choose **Upload**, then **Add files**.

   ii. Browse to and select the `IMG2.jpg` file on your computer.

   iii. Expand the **Properties** section, and configure the following:

- In the **Server-side encryption settings** section, choose **Specify an encryption key**.
- For **Encryption key type**, choose **AWS Key Management Service key (SSE-KMS)**.
- For **AWS KMS key**, select **Choose from your AWS KMS keys**.
- From the **Available AWS KMS keys** drop-down menu, choose `mk`.

Figure 7.7 shows the encryption options:

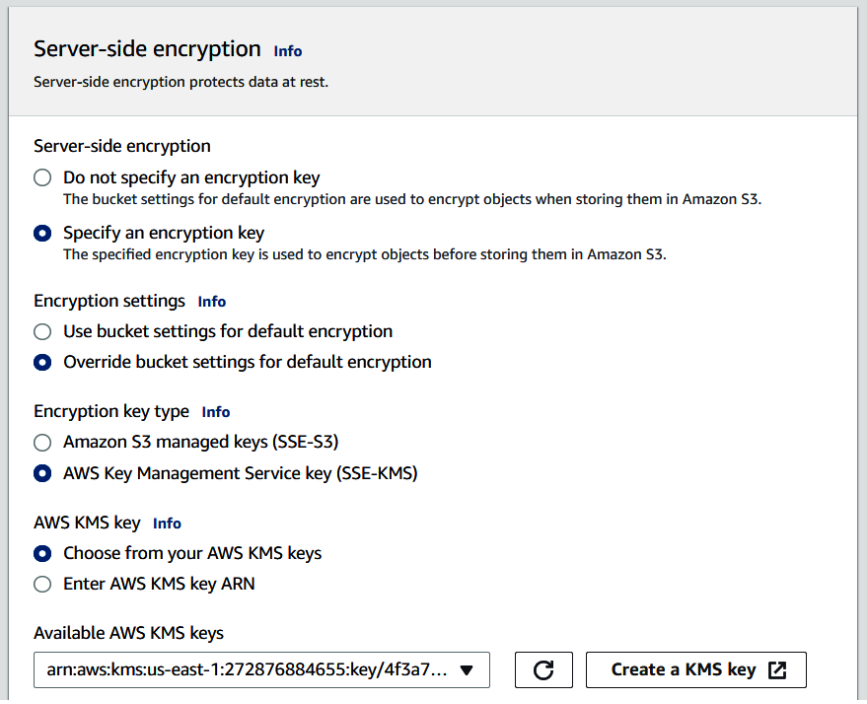


Figure 7.7 – Server side encryption

At the bottom of the page, choose **Upload**. Then click **Close** in the upper-right corner. Now, `IMG2.jpg` is listed as an object in the `data` bucket. Examine the object property and you will notice that server-side encryption using SSE-KMS is enabled on this object:

The following steps explain what transpired in the process of loading `IMG2.jpg` to `data`:

- The user requested to upload and store the `IMG2.jpg` file as an encrypted object in the data `bucket`.
- Amazon S3 requested a data key from AWS KMS to use to encrypt the file.
- AWS KMS generated a plaintext data key and encrypted the data key by using the customer-managed key `mk`.
- AWS KMS sent both copies of the data key to Amazon S3.
- Amazon S3 encrypted the object by using the plaintext data key, stored the object, and then deleted the plaintext data key. The encrypted key was kept in the object metadata.

When an object is stored in S3, the data key and the encrypted data (object) are both stored in S3. *Figure 7.8* gives an illustration.

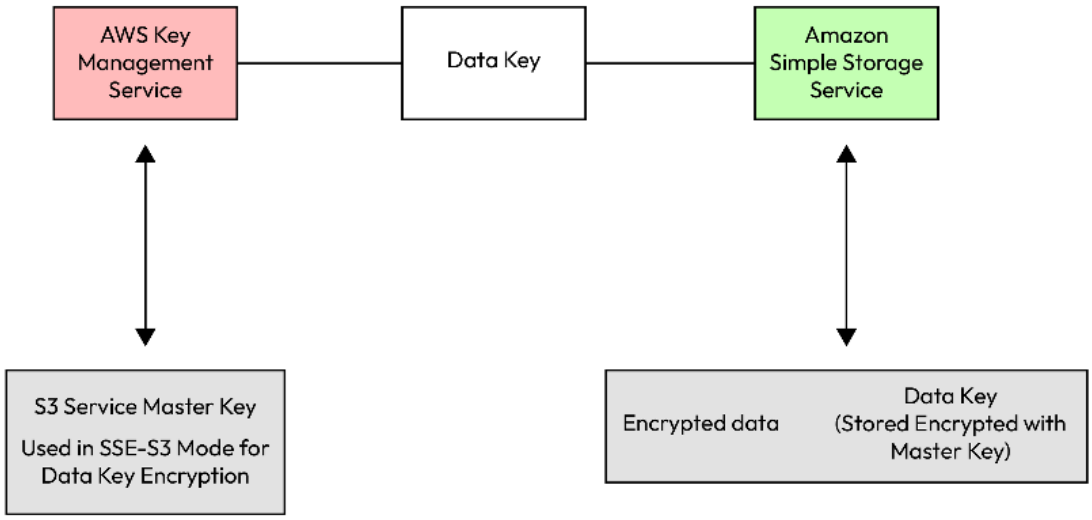


Figure 7.8 – Both data key and encrypted data are stored in S3

3. **Try to access the encrypted object**:

In the **Object overview** section at the top of the page, copy the object URL to your clipboard. Paste the object URL into a new browser tab and attempt to load the page. You will receive an **Access Denied** error, as shown in *Figure 7.9*:

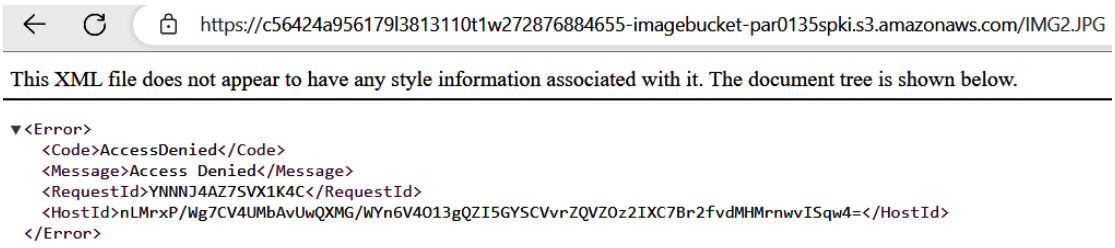


Figure 7.9 – Access Denied due to S3 bucket permissions

Now let's try to fix the problem by opening the S3 bucket and object permissions:

I.Go back to the S3 bucket and modify the public access permissions `bucket data`:

i.Choose the **Permissions** tab.

ii. In the **Block public access** section, choose **Edit**.

iii.Clear **Block all public access** and click **Save changes**.

iv.Also modify the access settings for the `IMG2.jpg` object:

- Choose the **Objects** tab and then select **IMG2.jpg**.
- Choose **Actions | Make public using ACL**.
- Choose **Make public**.

II.Return to the browser tab that you opened earlier and attempt to load the S3 object URL and refresh the page. You'll now see a different error as shown in *Figure 7.10*:

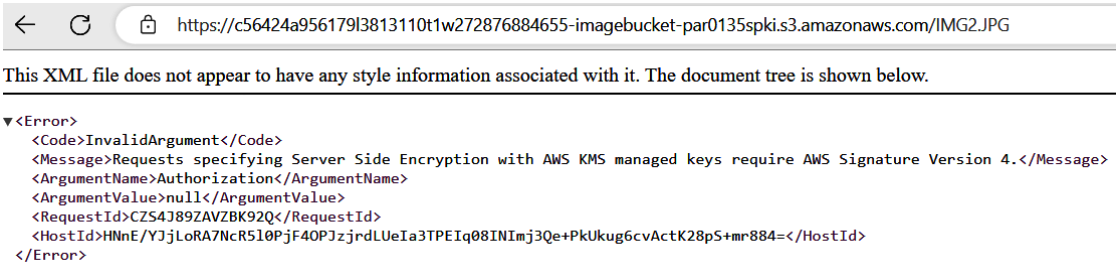


Figure 7.10 – Access Denied due to object encryption

4. **Access the encrypted object URL**:

In the Amazon S3 console, go to bucket `data`. On the **Objects** tab, select `IMG2.jpg`, and then choose **Open**. The image opens in a new tab or window. Why is that? If you analyze the URL that was used to successfully open the object, you will notice the format is something similar to the following:

```
https://c56424a956179l3813110t1w272876884655-imagebucket-
par0135spki.s3.us-east-1.amazonaws.com/IMG2.JPG?response-content
-disposition=inline&X-Amz-Security-Token=IQoJb3JpZ2luX2VjEC0aCXV
zLWVhc3QtMiJGMEQCIFlnR7JdNb%2BzVYg6onN9TAJnJbuBzkjTolw%2Frf%2BAA
4v8AiByWi%2BoCDH4NjN3kscEngRdMhS8PEQQxs6yWTEcMZFPair0Agj2%2F%2F%
2F%2F%2F%2F%2F%2F%2F%2F8BEAAaDDI3Mjg3Njg4NDY1NSIM%2BOXEy7qW4Ltud
kqZKsgCUDjCFWSoYDnEsuVFS3YzlARzGQnX%2FSNdy3EBYpXCw9cCsLuivYap9nn
f8FpYhTQpjw4Q5iHd7XZxBjsIFxwxWxftmM7GFR8pr3Bj33NCRckPshQ7%2FopYJ
%2FDulh5q3XOaXz5laSv6BzAfm1LTsXCRabgDwv0HKmsDvVcOfClt6%2BdJAuw2k
p5ipPNh1nA8F0GUyQly6w297lsF%2Bd8nblSqru4GFkuXQ90ioEm5B%2Fo3tDxDD
nOrzEXIXXfntnlcknr3Fov1e5TtrUKgln1BfOCnO0AdZvzgIncDTxsmpRonWWqK%
2FfWe7pMIX%2FxrSEpMZfHv5U6oACkEsm0nu6nfU2uOdXqbMfV1ijKowKY4DAdD0
goFJ1KBk%2BKi%2Fc86YIHndWjij%2BJlJe3Wxcf9sl%2F7fLpV5c3L9zM4sf1yb
rXZgiqRY7VB2PJzB3QTbzDuvv2gBjqIArm8e4LAAdY14psHogc%2FzaoF7xGd0Wy
9mLB9gWPru9wUmuDeP%2BS0737GQZXe50SvPGPuAUNOw1lxyAq87G04p62oKvkcF
n4j%2FEGKGHwVbdmtToVM%2BqlxtBbDBq%2BNoydhvBnUroXLx2fF2JyViodFPvM
YVJCvuAIRox9KKSvLH7oP8%2FuC2bm6cwjPGL04OhcftmKS%2BaRK0%2BcY0c9vN
%2FvkkNUswKsLoBoMDuso7TSBfXMEdxK5iW%2BQqORNKTs1Io051QyCqO0dvJ%2F
rJz5q0iki%2FqCNVUqP%2FQNPlUVrXvFtZVNJHh792jTVbE8KyUbBsu240uYPzr9
dbF%2FfKQlF%2FSgPey6rPbifBpnWuA%3D%3D&X-Amz-Algorithm=AWS4-HMAC-
SHA256&X-Amz-Date=20230325T212511Z&X-Amz-SignedHeaders=host&X-Amz
-Expires=300&X-Amz-Credential=ASIAT7CF22KX7N6F64VX%2F20230325%2
Fus-east-1%2Fs3%2Faws4_request&X-Amz-Signature=b8cbcea71f497aa8
9222025e7135026e6fc62d35e38bca020e479718fcc20624
```

The following steps explain what transpired in the preceding process of accessing `IMG2.jpg`:

- You requested to open an object.
- Next, Amazon S3 noticed that the requested object was encrypted. Because you were authenticated to the AWS account when using the Amazon S3 console, the signature version 4 authentication information was automatically included in the request.
- Amazon S3 then sent the encrypted copy of the data key that the object was encrypted with to AWS KMS.
- AWS KMS then decrypted the data key by using the AWS KMS key (which never leaves the AWS KMS service).
- AWS KMS then sent the plaintext data key back to Amazon S3.

Finally, Amazon S3 decrypted the ciphertext of the data object, allowed you to open the object, and deleted the plaintext copy of the data key.

5. **Verify the events from the CloudTrail event history**:

CloudTrail provides an audit log of API calls that are made in the AWS account, and its event history provides access to events from the last 90 days of account activity. Each time an API call is made to an AWS service within the Region that you have selected, if that service reports such events to CloudTrail, then the event and the AWS service that reported the event are listed.

In the search box to the right of **Services**, search for and choose **CloudTrail** to open the CloudTrail console. In the navigation pane, choose **Event history**.

Click on the drop-down menu on the left that currently displays **Read-only** and choose **Event source**. In the **Enter an event source** search box, search for `kms` and choose **kms.amazonaws.com**, as shown in *Figure 7.11*:

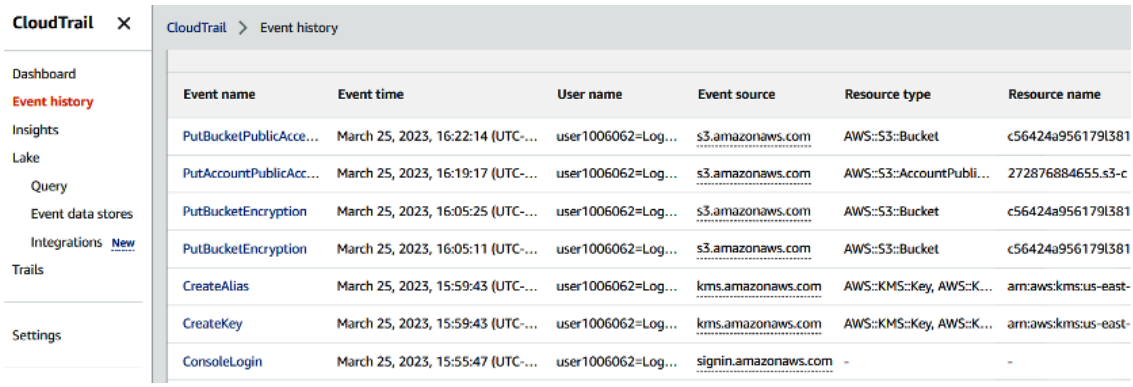


Figure 7.11 – CloudTrail event history

Analyze the `GenerateDataKey` event by choosing the **GenerateDataKey** link. In the **Event record** section, observe the details of the event, as shown in *Figure 7.12*:



Figure 7.12 – CloudTrail GenerateDataKey event

Use the same steps and analyze the **Decrypt** event by choosing the link for the **Decrypt** event name, as shown in *Figure 7.13*:

Analysis: This event was generated when you successfully opened the `img2.jpg` file from the Amazon S3 console. Notice that the record again details who made the request, which AWS KMS key provided the unencrypted data key back to Amazon S3, and which S3 object was decrypted with the plaintext data key.



Figure 7.13 – CloudTrail Decrypt event

In this section, we have dived into the encryption process of S3 object uploading and the decryption process of object opening. CloudTrail logs all AWS KMS API activity and helps to examine the past usage of AWS KMS keys.

In the next section, we will closely look at AWS logging and monitoring services, including AWS CloudTrail and others.

AWS logging, monitoring, and incident handling

Logging and monitoring are everywhere in the AWS cloud. As we discussed earlier, the third A in AWS cloud's AAA provides traceability of all activities in the cloud. We also introduced CloudTrail and analyzed the CloudTrail logs generated in the encryption and decryption process. In this section, we will explore further the AWS security services related to cloud resource logging and monitoring.

While **CloudTrail** provides a record of API calls made within a customer's AWS account, thus enabling customers to monitor and audit activity within their accounts, Amazon CloudWatch is a managed service that enables customers to monitor, store, access, and analyze log files from AWS services and their own applications, such as EC2 instances, RDS databases, Lambda functions, and more, to gain insights into the health and performance of their applications and infrastructure. Some of the most important features of the AWS cloud are elasticity and auto-scaling, and **CloudWatch** is the service that monitors unit utilization, sets threshold alarms, and integrates with other AWS services such as AWS Lambda and AWS EC2 to enable automatic scaling of resources based on application demand.

AWS Config is a service that allows users to assess, audit, and evaluate the configurations of their AWS resources. It provides a historical record of configuration changes to AWS resources and can be used to simplify compliance auditing and reporting. It can also be integrated with other AWS services such as AWS CloudTrail and AWS Lambda to enable automated remediation of non-compliant resources. AWS Config can be used to track changes to resource configurations over time and to troubleshoot issues by pinpointing the root cause of changes or issues.

AWS Trusted Advisor checks four cloud infrastructure pillars, including security, performance, resilience, and cost. These checks analyze a user's AWS infrastructure and provide recommendations for improving resource utilization, security, and cost efficiency. For example, Trusted Advisor can identify security vulnerabilities in AWS S3 configurations and suggest ways to improve security settings for S3 buckets and objects. In addition, Trusted Advisor detects AWS service limits in users' environments, such as the number of instances that can be launched in a region, and provides users with direct access to AWS technical support to increase the limit. It can also be configured to provide automatic notifications when changes or issues are detected in a user's AWS environment.

Amazon Inspector is a security assessment service used to scan vulnerabilities, by providing a list of security findings and recommendations to help customers remediate potential vulnerabilities. Amazon Inspector supports a wide range of assessments, including checks for common security issues such as insecure communication protocols, vulnerabilities in software libraries, and misconfigurations in network and application components. It also supports assessments for compliance with industry standards such as the **Payment Card Industry Data Security Standard** (**PCI DSS**) and the **Health Insurance Portability and Accountability Act** (**HIPAA**). For example, when an EC2 instance is compromised, we can isolate the instance and run Inspector against it to assess the vulnerability and security risks.

Amazon Macie scans storage to detect personal information such as Social Security numbers, including Amazon S3, Amazon RDS, and Amazon Redshift. It can also provide users with a detailed inventory of their data assets, including the location and risk level of sensitive data. Macie can also monitor data access patterns and alert users to potential data breaches or unauthorized access to sensitive data. It can also provide recommendations for improving data security, such as access control policies, encryption, and data retention policies.

AWS Shield is a managed service that provides protection against **Distributed Denial of Service** (**DDoS**) attacks. While AWS Shield Standard is a free service that is automatically enabled for all AWS customers and provides protection against most common network and transport layer DDoS attacks, including SYN/ACK floods, UDP floods, and reflection attacks, Shield Advanced is a paid service that provides additional protection against more sophisticated DDoS attacks, including application layer attacks. AWS Shield Advanced includes 24/7 access to the AWS **DDoS Response Team** (**DRT**), for assistance in mitigating DDoS attacks.